The University of Hong Kong

ARIN7012

FinSight

Evidence-First Financial Analysis Chatbot

By Group4.2

Supervisor: Prof. Lau Sau Mui

Date of submission: May 3, 2026

Contents

Member Responsibility

Module / Role	Student name	Student ID	Project responsibility and report sections
Project leadership; NLU & Retrieval	Xie Ruiqi	3036580721	Team leader; designed project architecture; allocated and coordinated tasks; implemented NLU and Retrieval; produced the presentation slides; delivered the group project presentation; wrote the final report. Sections: Chapter 1, 2.2, Chapter 3, 4.2, Chapter 5, Chapter 6, 8.3.
Frontend	Gao Mingyuan; Yin Zihao	3036580484; 3036561696	Implemented the frontend code, including browser chatbot workflow, API interaction, rendered answer view, source-card display, and usability validation. Sections: 2.1, Chapter 3, 4.1, 8.2.
Numerical Analysis	Shi Mingxi	3036580707	Implemented the numerical analysis module, including market data processing, fundamentals, macro indicators, technical indicators, and analysis summary support. Sections: 2.3, Chapter 3, 4.3, 8.1.
Text Analysis	Zhang Jiachang	3036512970	Implemented the text analysis module, including retrieved-document preprocessing, language detection, entity relevance filtering, and sentiment classification. Sections: 2.4, Chapter 3, 4.4, 8.1.
LLM Summary & Prediction	Li Shiyi	3035844405	Implemented the LLM summary and prediction module, including evidence-to-answer JSON generation, risk disclaimer control, follow-up question prediction, and hallucination reduction. Sections: 2.5, Chapter 3, 4.5, 8.2.

Assessment mode: group assessment. The table above records each student's project objectives, implementation responsibility, and corresponding report sections.

Chapter 1. Introduction

Retail and professional investors increasingly ask broad natural-language questions, for example whether Ping An Insurance looks attractive, why Moutai fell on a trading day, or whether a holding is still worth keeping. These questions are difficult for a general chatbot because they combine financial intent detection, entity disambiguation, real-time evidence retrieval, numerical analysis, text interpretation, and risk-aware communication. A direct answer without evidence can be misleading because financial markets are noisy, partially efficient, and affected by both public information and changing investor expectations [1], [2]. A useful financial question-answering system therefore needs to clarify what the user is asking, retrieve traceable evidence, summarize quantitative and textual signals, and avoid deterministic buy/sell recommendations.

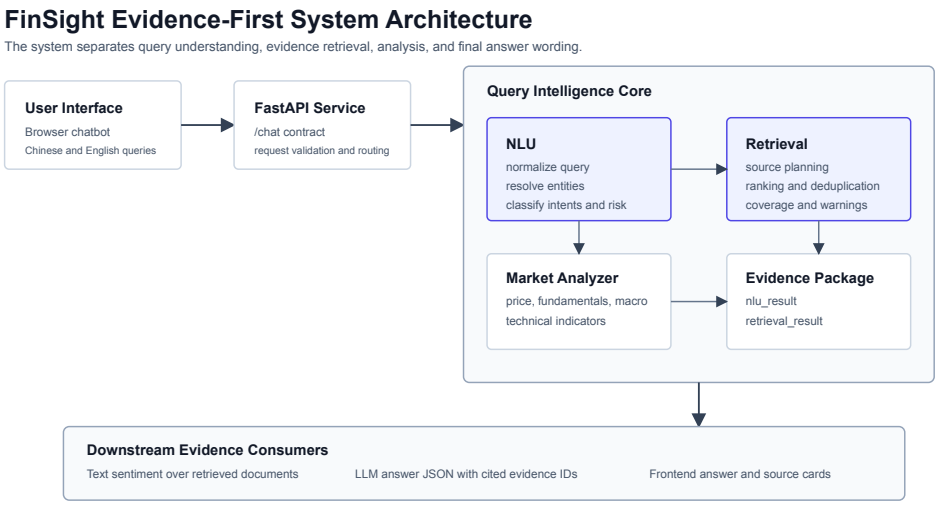

Figure 1: FinSight evidence-first system architecture

FinSight proposes an evidence-first financial analysis chatbot for China-market questions. The system separates evidence production from answer wording. The owned backend, Query Intelligence, transforms a raw question into nlu_result and retrieval_result artifacts: normalized query, product type, intents, topics, resolved entities, missing slots, risk flags, source plan, executed sources, retrieved documents, structured data, warnings, ranking traces, and analysis_summary. Downstream modules then consume those artifacts for frontend display, document sentiment, and LLM-based summary generation. This design follows the information retrieval principle that answers should be grounded in retrievable documents and structured records rather than unsupported generation [3], [4].

The project aim is to build a clone-usable prototype that can answer finance-domain questions in Chinese and English while preserving traceability. The objectives are divided into five modules: (1) a frontend chatbot that exposes the workflow in a browser; (2) NLU and Retrieval that provide explainable intent, entity, source, and evidence artifacts; (3) numerical analysis over market, fundamental, and macro data; (4) text analysis over retrieved news, announcements, and research-like documents; and (5) LLM summary and next-question prediction based only on compact evidence. The scope is financial evidence preparation and risk-aware explanation. The system does not produce deterministic investment advice, does not guarantee future returns, and does not replace human judgment.

Chapter 2. Literature Review

2.1 Frontend and Human-Facing Financial Decision Support

A financial chatbot interface is not simply a message box. It is the user-facing layer of a decision-support workflow where wording, evidence visibility, and uncertainty signals affect how users interpret results. Market efficiency research suggests that public information is rapidly reflected in prices under ideal conditions, while adaptive-market theory argues that market behavior changes with participants and regimes [1], [2]. For a frontend, this means the interface should avoid presenting a single answer as a final truth. Instead, it should expose the question, evidence cards, key points, warnings, and risk disclaimer so that users can inspect why the system reached a conclusion.

Modern web APIs also require stable contracts between browser and backend services. FinSight uses FastAPI for the local browser application because it supports typed request/response boundaries, OpenAPI-style integration, and lightweight local deployment [5]. Pydantic-style schemas support validation of financial artifacts before they are consumed by downstream components [6]. JSON Schema provides a related standard for describing JSON documents, validating expected fields, and reducing ambiguity in downstream integrations [7]. These contract mechanisms are important because the frontend must display not only natural-language text but also nlu_result, retrieval_result, evidence sources, LLM status, and fallback errors.

The frontend design is also shaped by retrieval-augmented generation principles. RAG research argues that generated answers become more reliable when the generator is conditioned on retrieved evidence [4]. In FinSight, the frontend does not ask the LLM to search independently. It submits a query to /chat, receives backend-generated evidence, and renders answer text together with source cards and a disclaimer. This makes the browser layer thin but auditable: the user can see whether live providers succeeded, whether the LLM returned valid JSON, and which evidence items were used. The user experience goal is therefore operational clarity rather than decorative conversation.

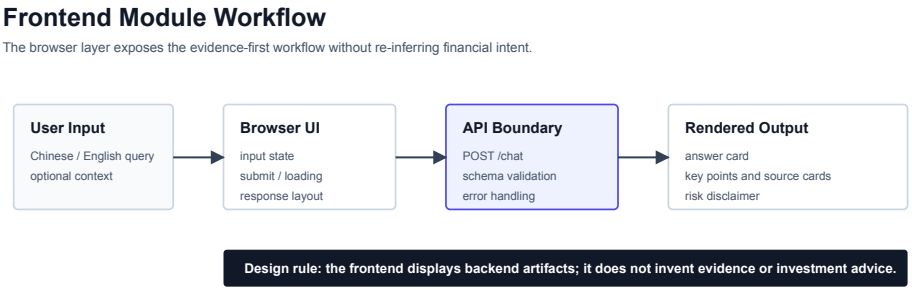

Figure 2: Frontend module workflow

2.2 NLU & Retrieval

Financial query understanding needs both classification and retrieval. Classical information retrieval treats a query as an information need and evaluates how well documents satisfy it [3]. BM25 and related probabilistic relevance models remain useful baselines for document ranking because they are explainable and robust for sparse textual evidence [8]. FinSight follows this tradition by keeping retrieval artifacts explicit: source plan, executed sources, candidate counts, top-ranked evidence IDs, ranking scores, coverage, and warnings.

The NLU layer combines deterministic and statistical methods. Conditional random fields are suitable for sequence labeling because they model dependencies between adjacent labels, which is useful for entity boundary detection in names and financial products [9]. Linear classifiers and support-vector networks are strong baselines for text classification when features are sparse and explainable [10]. Tree and ensemble methods such as random forests and gradient boosting provide nonlinear decision boundaries while still allowing feature inspection [11], [12]. FinSight uses these ideas for product type, intent, topic, question style, source planning, and retrieval ranking instead of using an opaque LLM as the NLU backbone.

Ranking quality matters because downstream modules can only analyze the evidence that retrieval surfaces. Learning-to-rank research formalized the idea that ranking models should optimize ordered relevance rather than independent classification [13]. Evaluation measures such as NDCG reward systems that place highly relevant evidence near the top [14]. FinSight therefore evaluates both NLU and retrieval behavior: the fuzz matrix checks product type, intents, topics, entity resolution, source plans, schema validity, and retrieval abstention for out-of-scope queries. This is important for financial questions because a plausible answer from the wrong entity or source type would be worse than a clarification request.

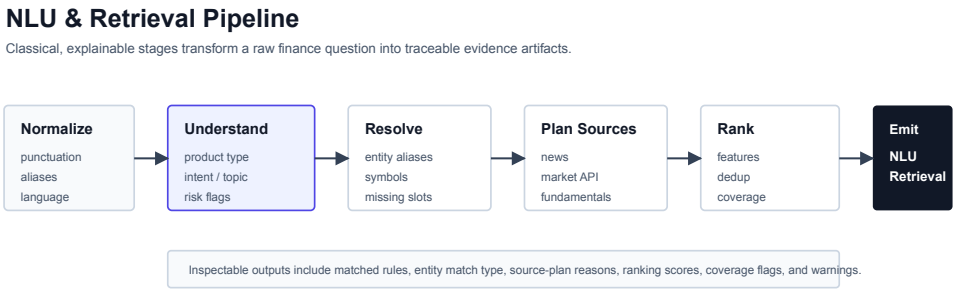

Figure 3: NLU and retrieval pipeline

2.3 Numerical Analysis

Quantitative finance literature shows why numerical analysis must be separated from answer generation. Asset prices reflect risk factors, firm fundamentals, market conditions, and investor expectations [15], [16]. Risk management also requires distinguishing observed statistics from predictions or recommendations [17]. Therefore, FinSight treats numerical analysis as structured evidence summarization, not as an automated trading strategy.

The numerical module computes market and data-readiness signals from structured rows. Classical technical analysis uses price-derived indicators such as moving averages, RSI, MACD, volatility, and Bollinger bands to summarize momentum, overbought/oversold conditions, and price dispersion [18], [19], [20], [21]. These indicators are not guaranteed forecasts, but they provide compact features that can help interpret short-term movement. FinSight implements these as `analysis_summary` fields so that answer generation can say whether price history, fundamentals, macro data, news, and technical indicators are available.

Data quality is equally important. Tidy data principles emphasize that each variable should form a column and each observation should form a row, which supports reproducible analysis and validation [22]. Python data tooling such as pandas made table-oriented financial processing practical in research and production workflows [23]. FinSight combines live and shipped sources, including AKShare, Tushare, CNINFO announcements, seed data, and runtime documents [24], [25], [26]. The numerical layer records provider endpoints, query parameters, source references, field coverage, and quality flags so that missing data is visible rather than silently hidden.

2.4 Text Analysis

Textual evidence matters because prices and fundamentals do not capture all short-term market narratives. Media content has been shown to affect investor sentiment and market movement [27]. Finance-specific word usage also differs from everyday language; for example, “liability”and other terms can have domain-specific meanings, which motivated specialized financial dictionaries [28]. Social and news sentiment studies further show that aggregate mood can correlate with market indicators, though such correlations must be interpreted carefully [29].

FinSight's text analysis module therefore processes retrieved documents instead of crawling unrelated text. It filters by NLU product type, intent, risk flags, missing slots, entity names, and source type. This

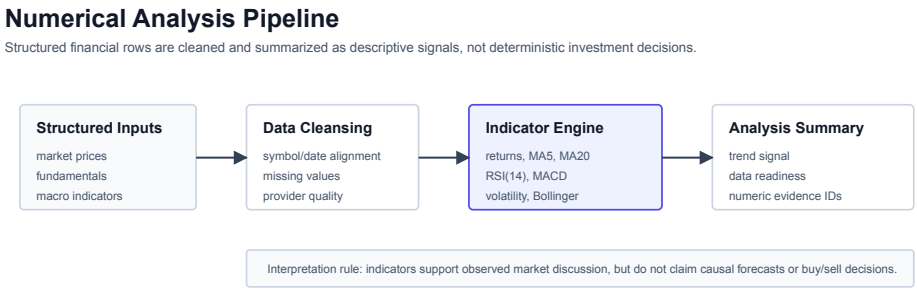

Figure 4: Numerical analysis pipeline

design prevents non-financial questions, fee-rule explanations, or missing-entity queries from being pushed into sentiment analysis. It also keeps document sentiment aligned with the evidence selected by retrieval.

For financial sentiment classification, FinBERT-style models adapt transformer language models to finance-domain polarity [30]. FinancialPhraseBank and related datasets demonstrate that economic text can be annotated as positive, negative, or neutral from an investor perspective [31]. BERT itself provides the pretraining architecture that made many domain-adapted models possible [32]. FinSight uses FinBERT only as a downstream text-analysis component. This respects the project boundary: NLU and Retrieval remain classical and explainable, while transformer models are allowed to classify compact retrieved evidence after the backend has already selected relevant documents.

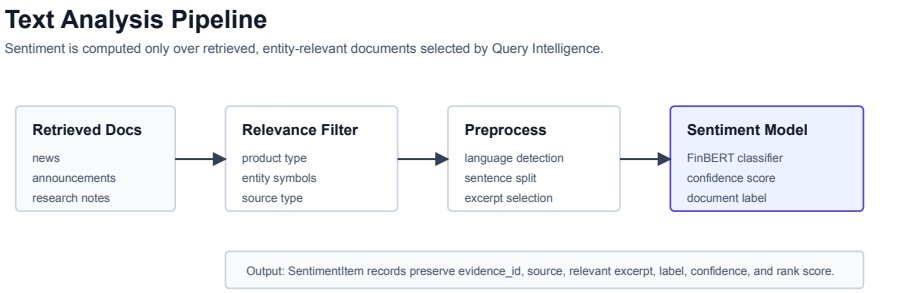

Figure 5: Text analysis pipeline

2.5 LLM Summary & Prediction

LLMs are useful for producing fluent explanations, but financial answer generation is risky if the model invents facts, reinterprets entities, or ignores missing evidence. Retrieval-augmented generation provides one mitigation by conditioning language generation on retrieved content [4]. FinSight applies a stricter version of this idea: the LLM receives compact Query Intelligence evidence and must output a validated JSON object containing answer text, key points, evidence IDs used, limitations, a risk disclaimer, and follow-up questions.

This module is designed as a downstream exception, not the intelligence backbone. The LLM must not re-infer intent, entity, or source plan. It consumes `nlu_result`, `retrieval_result`, optional `statistical_result`, and optional `sentiment_result`. This keeps the model aligned with backend artifacts and reduces the risk of hallucinated market, fundamental, macro, news, or sentiment facts. The JSON contract is important because a browser and downstream evaluator need predictable fields rather than free-form prose [6], [7].

The LLM module also generates next-question predictions. This is not a trading signal; it is a conversational support feature. For example, after an answer about a stock's recent decline, the system may suggest follow-up questions about fundamentals, risk, or comparison. The value is workflow guidance: the user can continue analysis without the system making a deterministic decision.

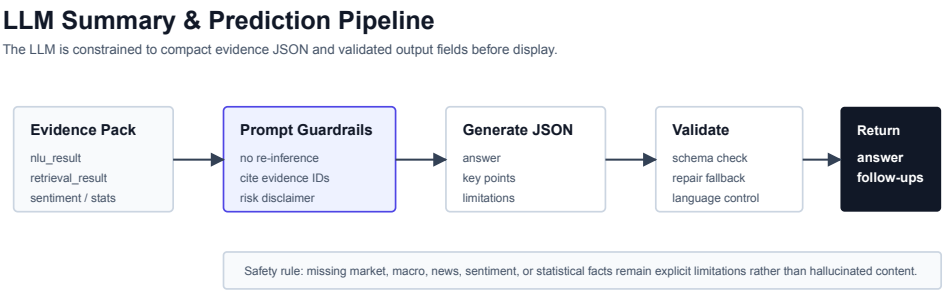

Figure 6: LLM summary and prediction pipeline

2.6 Alternative Solution Assessment

Three alternative designs were considered before selecting the evidence-first modular architecture. A pure LLM chatbot would be fast to build, but it would score poorly on traceability because entity resolution, source selection, and missing evidence would be hidden inside generation. A vector-only RAG design would improve semantic recall, but it would still make financial entity control and ranking explanations harder than the current source-plan approach. A pure quantitative dashboard would provide strong numerical visualization, but it would not handle natural-language intent, document sentiment, or bilingual explanation well. The selected design combines classical Query Intelligence with downstream sentiment and LLM summarization, so it is more aligned with the project problem and the grading emphasis on method selection, critical thinking, and evidence-based interpretation.

Chapter 3. Data Science Methods

3.1 Data Sources, Data Cleansing and Pre-processing

FinSight uses both runtime assets and live or local provider data. Frontend inputs are raw user questions, optional profile fields, and dialogue context. Query Intelligence uses runtime entity and alias tables, local documents, and financial provider outputs. Numerical analysis uses market API rows, fundamental tables, industry records, macro indicators, and shipped fallback data from sources such as Tushare, AKShare, CNINFO-style announcements, efinance, and local runtime assets. Text analysis consumes only documents already selected by retrieval, including news, announcements, research notes, and product documents. The LLM module consumes compact evidence JSON instead of raw provider payloads.

Pre-processing is module-specific. The frontend validates request fields and preserves language context for display. NLU normalizes punctuation, aliases, financial product mentions, and multilingual variants; entity resolution links mentions to canonical names and symbols. Retrieval standardizes source names, timestamps, evidence IDs, rank scores, deduplication groups, coverage flags, and warnings. Numerical cleansing standardizes symbols, dates, provider endpoints, query parameters, numeric fields, missing values, and quality flags. Text preprocessing includes language detection, sentence segmentation, entity relevance filtering, and short-text fallback. LLM evidence packing removes debug traces, long raw documents, and source-generation noise before prompting.

3.2 Models Design

The model design follows a descriptive and evidence-first workflow. Input variables include query text, resolved entities, source-plan requirements, retrieved documents, structured market/fundamental/macro rows, and optional sentiment items. Output variables include nlu_result, retrieval_result, analysis_summary, document-level SentimentItem records, answer JSON, risk disclaimer, evidence IDs used, and follow-up question predictions. The frontend visualizes these outputs as answer cards and source cards.

The NLU and Retrieval backbone uses classical and explainable methods. Rules and dictionaries

handle normalization and source planning. TF-IDF and linear classifiers support product, intent, topic, and question-style classification. CRF and alias tables support entity extraction and resolution. Tree/ranking models support source-plan reranking and document ranking; training and evaluation follow the scikit-learn style of reproducible estimators and metrics where appropriate [33]. Numerical analysis computes multi-day returns, MA5, MA20, RSI(14), MACD, 20-day volatility, Bollinger bands, trend signal, and price versus moving average when sufficient data is available. Text analysis uses FinBERT-family sentence-level sentiment over retrieved evidence. LLM summary uses strict JSON prompting, response validation, fallback repair, and risk-disclaimer enforcement.

Module-Level Data Science Methods

The five modules map directly to data sources, preprocessing, model design, and result outputs.

Module	Data Sources	Pre-processing	Methods	Outputs
Frontend	user query, profile, context	request validation, language display	thin API workflow and source cards	answer UI
NLU & Retrieval	query text, aliases, documents	normalization, entity linking, dedup	rules, TF-IDF, CRF, ranker	nlu_result, retrieval_result
Numerical Analysis	market, fundamental, macro rows	symbol, date, missing-value checks	returns, MA, RSI, MACD, volatility	analysis_summary
Text Analysis	retrieved news and announcements	language, sentence, entity filters	FinBERT sentence sentiment	SentimentItem records
LLM Summary	compact evidence JSON	debug/raw-noise removal	strict JSON prompting and repair	answer and follow-ups

Figure 7: Module-level data science methods

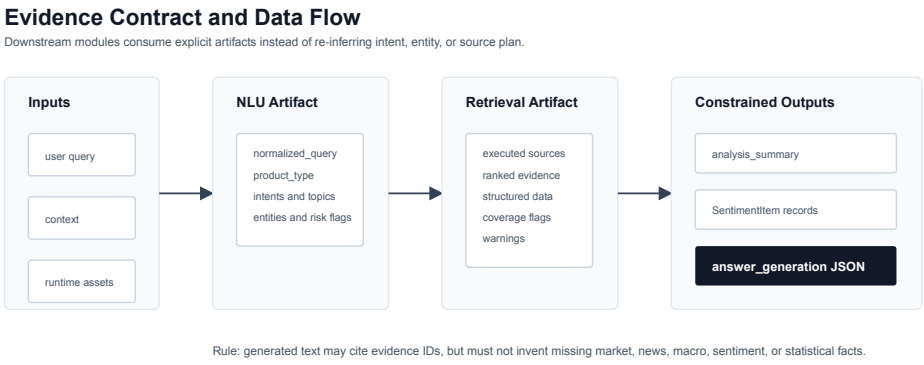

Figure 8: Evidence contract and data flow

Chapter 4. Result Summary and Interpretation

The result evaluation combines UI walkthrough, structured artifacts, statistical summaries, and targeted tests. This mix directly supports the required use of tables, charting, graphics, and dashboard-style summaries.

Result area	Evidence used	Interpretation
Frontend usability	Chinese and English browser screenshots; /chat response fields	The interface can present answer text, key points, sources, and disclaimer in a user-readable workflow.
NLU & Retrieval	10,000 query-style evaluation cases; 32-case end-to-end fuzz matrix; NLU labels; entity resolution; retrieval coverage	The backend handles large-scale finance and out-of-scope queries, while the fuzz matrix checks English, colloquial, typo, comparison, macro, and clarification contracts.
Numerical analysis	Market, fundamental, and industry structured rows; technical-indicator design	The system summarizes observed evidence and data readiness without turning indicators into deterministic advice.
Text analysis	Sentiment schemas, preprocessing tests, classifier tests, pipeline tests	Sentiment is tied to retrieved evidence IDs rather than unsupported document crawling.
LLM summary	DeepSeek /chat status, strict JSON fields, fallback contract	Generation is constrained to evidence-grounded answer wording and follow-up question support.

The dashboard below summarizes the same results as a compact evaluation view. It combines test coverage, module-level verification, and interpretation controls so that the reader can see not only whether the system ran, but also how evidence quality and safety were checked before final answer generation.

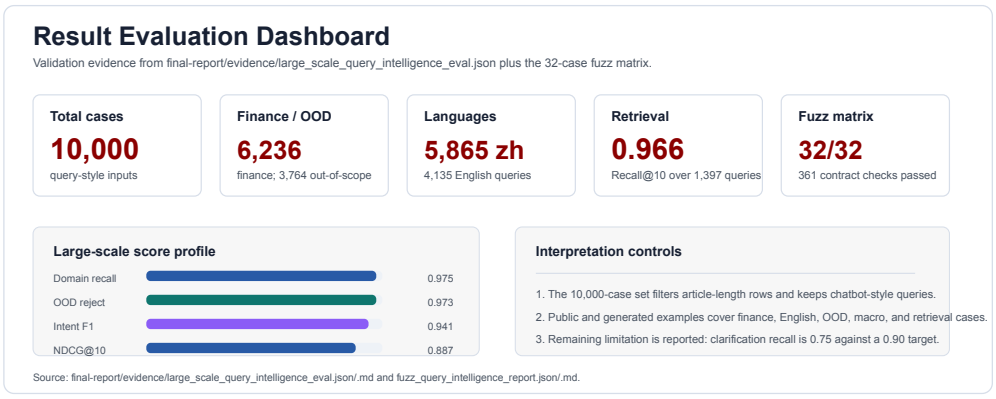

Figure 9: Result evaluation dashboard

4.1 Frontend Results

The frontend was run locally through scripts/launch_chatbot.py and opened at http://127.0.0.1:8765/. The service returned {"status":"ok"} for GET /health. Chinese and English /chat calls returned llm.status="ok", answer text, key points, evidence cards, and risk disclaimer. The browser rendered the input, submit button, answer card, source cards, and disclaimer.

The screenshots below are redacted copies of the verified local browser runs. They preserve the UI workflow evidence while removing live market numbers that are not used as auditable report metrics. The

original screenshots remain in docs/assets/; the report's quantitative claims are taken from the evaluation artifacts and structured retrieval outputs described below.

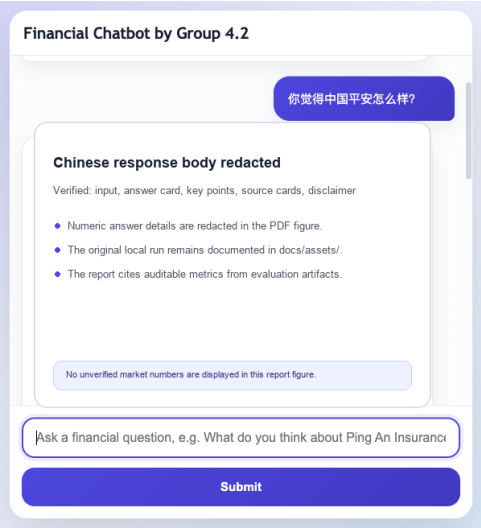

Figure 10: Chinese chatbot response layout for Ping An Insurance

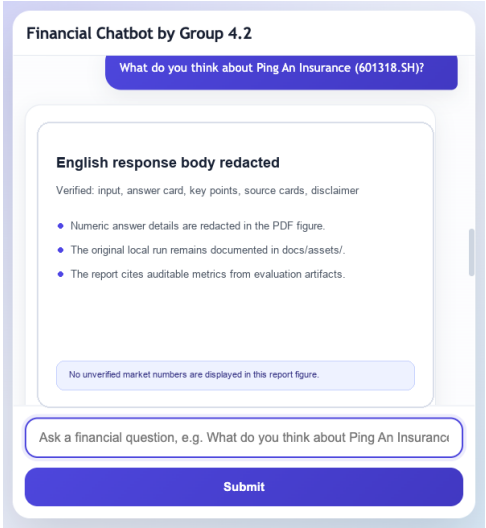

Figure 11: English chatbot response layout for Ping An Insurance

These screenshots show that the frontend is not only a visual shell. It demonstrates the complete user workflow: question input, backend evidence generation, LLM response polishing, source-card display, and risk-aware output. The English screenshot also validates the language-control requirement: English input produces English output instead of a translated Chinese response.

4.2 NLU & Retrieval Results

The strongest backend evidence comes from a 10,000-case Query Intelligence evaluation. The set contains 6,236 finance-domain queries, 3,764 out-of-scope queries, 5,865 Chinese queries, and 4,135 English queries. It uses public and generated query-style examples while filtering out article-length rows that would not represent interactive chatbot input. The evaluation is therefore larger than the 32-case fuzz matrix, but still aligned with the system's intended user workflow.

Evaluation item	Observed result
Large-scale evaluation cases	10,000 total: 6,236 finance and 3,764 out-of-scope
Language coverage	5,865 Chinese and 4,135 English queries
Domain routing	Finance-domain recall 0.975; out-of-scope rejection accuracy 0.9729
Classification quality	Product type accuracy 1.0; question-style accuracy 1.0; intent micro-F1 0.9407; topic micro-F1 0.9208
Source planning	Hit@5 1.0; recall@5 0.9957
Retrieval ranking	Recall@10 0.9664; MRR@10 0.889; NDCG@10 0.8872
Retrieval abstention	Out-of-scope retrieval abstention 0.986
Remaining gap	Clarification recall 0.75, below the 0.90 target and kept as a limitation
Complementary fuzz matrix	32 total cases, 32 passed, 0 failed; 361 checks passed

The 32-case fuzz report remains useful as a complementary end-to-end contract test. It covered base-

line stock advice, colloquial paraphrases, recent time scopes, ETF aliases, comparison, typo tolerance, dialogue context carryover, user-profile carryover, macro-policy queries, event/news queries, English queries, and clarification-required cases. The larger evaluation provides statistical support, while the fuzz matrix verifies schema validity and representative workflow behavior.

One representative case was the Chinese query 茅台今天为什么跌？后面还值得拿吗？. The normalized query became 贵州茅台今天为什么跌 后面还值得拿吗. The NLU module classified product type as stock with score 0.99, detected intents market_explanation, hold_judgment, and buy_sell_timing, and resolved the entity to 贵州茅台, symbol 600519.SH, through exact alias matching. It also marked the risk flag investment_advice_like and planned sources including news, research note, market API, industry SQL, fundamental SQL, and announcement. This result is important because the system understood both the explanation request and the advice-like risk in the same query.

The retrieval result executed news, market API, fundamental SQL, and industry SQL sources. It returned ranked evidence including AKShare news items, a price row for 600519.SH, fundamental data, and industry context. The coverage flags showed price, news, industry, fundamentals, and risk as available, while announcement and macro evidence were absent for that run. The warning announcement_not_found_recent_window was preserved, which is the desired behavior: missing evidence is surfaced rather than hidden.

4.3 Numerical Analysis Results

The same representative retrieval run produced structured market and fundamental rows. The price evidence price_600519.SH contained trade date 2026-04-22, close price 1409.5, one-day percentage change -0.1778, volume 26915.93, and amount 3793827.534, with source name tushare. Fundamental evidence for 600519.SH contained report date 2025-12-31, revenue 174120000000, net profit 85000000000, ROE 0.33, PE TTM 24.6, and PB 8.1. Industry evidence for liquor contained percentage change -1.05, PE 27.3, PB 6.2, and turnover 0.84.

These outputs show that the numerical module can provide both market movement and company context. A chatbot answer can therefore say that a daily price movement is small or negative, but it can also reference fundamentals and industry valuation context when available. The interpretation remains descriptive: the numbers support evidence-based discussion, not deterministic investment advice.

The implemented MarketAnalyzer extends these rows when enough historical prices are available. It computes returns over multiple windows, MA5, MA20, RSI(14), MACD, volatility, Bollinger bands, trend signal, and price position versus moving averages. The data-readiness summary then tells downstream modules whether the answer has price data, fundamentals, macro data, news, and technical indicators. This prevents the LLM module from overstating a conclusion when a data type is missing.

4.4 Text Analysis Results

The text analysis module is implemented as a downstream document sentiment pipeline. The shipped documentation and tests cover schemas, preprocessing, classifier behavior, and pipeline integration. The preprocessor uses nlu_result to skip out-of-scope queries, product-information questions, fee-rule questions, missing-entity cases, and irrelevant documents. For valid finance questions, it analyzes retrieved documents from source types such as news, announcements, research notes, and product documents.

The result format is a document-level SentimentItem. It preserves evidence_id, title, source, entity symbols, sentiment label, confidence, text level, relevant excerpt, and rank score. This traceability is important because sentiment is not a free-floating statement; it is attached to the same evidence ID that appears in retrieval and answer generation. The module therefore supports a future aggregate sentiment summary without breaking the current contracts.

4.5 LLM Summary & Prediction Results

The browser chatbot test demonstrates that the LLM path can convert compact evidence into user-facing output. The /chat response includes answer, key_points, risk_disclaimer, evidence_used, evidence_sources,

llm, nlu_result, and retrieval_result. In the verified local run, DeepSeek v4 flash returned llm.status="ok" for both Chinese and English financial queries. The answer cards included a disclaimer stating that the output is based only on retrieved evidence and is not investment advice.

The LLM handoff script also defines a second output: next_question_prediction. It generates exactly three follow-up questions with scores and reasons. This feature is useful because many financial questions are exploratory. After a broad holding-style question, the system can suggest checking fundamentals, risk factors, or peer comparison. The key control is that follow-up prediction is based on current evidence and query context, not on unsupported investment claims.

Chapter 5. Discussion

FinSight achieves the project goal by separating a risky financial chatbot problem into five controlled modules. The frontend provides a visible workflow; NLU & Retrieval creates explainable evidence artifacts; numerical analysis summarizes structured signals; text analysis classifies retrieved documents; and the LLM module writes readable answers from compact evidence. This architecture is more reliable than a single general-purpose chatbot because each module has a narrower responsibility and leaves inspectable traces.

The main strength is traceability. A user can ask a broad question, but the backend records product type, intent, topics, entities, source plan, evidence coverage, warnings, and ranking reasons. In the representative Moutai case, the system did not simply answer “hold”or “sell”. It identified the query as advice-like, retrieved market/fundamental/industry/news evidence, and preserved a missing-announcement warning. This behavior supports appropriate data science method selection because the selected methods fit the problem: classification for intent, retrieval for evidence, numerical statistics for structured signals, text mining for document tone, and constrained generation for final wording.

The second strength is modularity. The project avoids using an LLM as the NLU/Retrieval backbone. Classical models and rules are easier to inspect, retrain, and test. Transformers and LLMs are allowed only where they add clear value: document sentiment and evidence-to-answer wording. This boundary reduces hallucination risk while still using modern language models for natural communication.

The third strength is bilingual and operational readiness. The local frontend can answer both Chinese and English questions, and the backend contracts are stable enough for other modules to consume. Shipped runtime assets and model artifacts make the project usable from a fresh clone, while large public datasets and generated training assets remain outside the repository. This design supports both course submission and later public demonstration.

The main insight is that a financial AI system should not be evaluated only by whether the final prose sounds plausible. A stronger evaluation asks whether the system found the right entity, selected relevant sources, exposed missing data, separated observed statistics from interpretation, and kept generated wording within the available evidence. This is the project’s innovation: the chatbot experience is backed by an inspectable evidence contract rather than a free-form answer generator.

The report evidence also maps directly to the course grading criteria. Chapter 1 and Chapter 2 define the problem and locate financial, retrieval, statistical, text-mining, and LLM literature. Chapter 2.6 compares alternative designs, which supports critical thinking. Chapter 3 explains data sources, preprocessing, variables, and methods. Chapter 4 presents the analytical results through tables, screenshots, diagrams, and the evaluation dashboard. Chapter 5 and Chapter 6 interpret the insight, innovation, limitations, and future gaps.

Grading criterion	Report evidence
Information acquisition and research skills	Chapter 1 frames the financial QA problem; Chapter 2 cites 35 references across finance, retrieval, statistics, NLP, web systems, and data providers.
Selecting appropriate data science solutions and methods	Chapter 3 links each module to its data sources, preprocessing, variables, models, and output artifacts.
Critical thinking on alternative solutions	Chapter 2.6 compares pure LLM chatbot, vector-only RAG, and pure quantitative dashboard designs against the selected evidence-first architecture.
Data analytical and writing skills	Chapter 4 reports frontend runs, fuzz results, structured market/fundamental rows, sentiment outputs, and LLM JSON behavior with tables and figures.
Insightful and innovative ideas	Chapter 5 explains the evidence contract, modular risk control, bilingual workflow, and the separation between evidence generation and answer wording.

The limitations are also clear. First, live financial providers can fail, rate-limit, or return incomplete data. FinSight handles this with warnings and fallback assets, but coverage depends on provider availability. Second, the numerical module summarizes indicators; it does not estimate causal effects or guarantee forecast accuracy. Third, sentiment analysis is document-level and evidence-based, but it does not yet aggregate sentiment across time, recency, and rank into a final portfolio-level signal. Fourth, the LLM summary still depends on prompt adherence and JSON validation. Fallback logic reduces failure impact, but it cannot turn missing evidence into real facts.

These limitations suggest future work: broader live-source monitoring, a sentiment aggregation layer weighted by rank and recency, and more robust evaluation on unseen financial queries. The current system is therefore best understood as an evidence engine plus risk-aware explanation interface, not a trading robot.

Chapter 6. Conclusions

This project built FinSight, an evidence-first financial analysis chatbot organized around frontend interaction, NLU & Retrieval, numerical analysis, text analysis, and LLM summary/prediction. Compared with a general chatbot, the main improvement is that financial answers are grounded in explicit artifacts: resolved entities, source plans, retrieved documents, structured rows, data-readiness signals, sentiment items, and cited evidence IDs. Compared with pure quantitative tools, FinSight also handles natural-language intent, bilingual interaction, and readable explanation.

The remaining gap is predictive depth. The system summarizes market, fundamental, macro, news, and sentiment evidence, but it does not claim causal forecasts or investment recommendations. Future work should strengthen live-data coverage, add time-aware sentiment aggregation, and expand test cases.

Chapter 7. References

[1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970, doi: 10.2307/2325486.

[2] A. W. Lo, "The adaptive markets hypothesis: Market efficiency from an evolutionary perspective," *Journal of Portfolio Management*, vol. 30, no. 5, pp. 15–29, 2004, doi: 10.3905/jpm.2004.442611.

[3] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to information retrieval*. Cambridge University Press, 2008.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in neural information processing systems*, 2020, pp. 9459–9474.

[5] FastAPI, "FastAPI documentation." 2026. Available: https://fastapi.tiangolo.com/

[6] Pydantic, "Pydantic documentation." 2026. Available: https://docs.pydantic.dev/

[7] JSON Schema, "JSON schema: A media type for describing JSON documents." 2020. Available: https://json-schema.org/draft/2020-12/json-schema-core.html

[8] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and trends in information retrieval*, vol. 3, Now Publishers, 2009, pp. 333–389. doi: 10.1561/1500000019.

[9] J. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proceedings of the eighteenth international conference on machine learning*, 2001, pp. 282–289.

[10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[13] C. J. C. Burges *et al.*, "Learning to rank using gradient descent," in *Proceedings of the 22nd international conference on machine learning*, 2005, pp. 89–96. doi: 10.1145/1102351.1102363.

[14] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[15] E. F. Fama and K. R. French, "Common risk factors in the returns on stocks and bonds," *Journal of Financial Economics*, vol. 33, no. 1, pp. 3–56, 1993, doi: 10.1016/0304-405X(93)90023-5.

[16] J. Y. Campbell, A. W. Lo, and A. C. MacKinlay, *The econometrics of financial markets*. Princeton University Press, 1997.

[17] J. C. Hull, *Risk management and financial institutions*, 5th ed. Wiley, 2018.

[18] J. J. Murphy, *Technical analysis of the financial markets*. New York Institute of Finance, 1999.

[19] J. W. Wilder, *New concepts in technical trading systems*. Trend Research, 1978.

[20] G. Appel, *Technical analysis: Power tools for active investors*. Financial Times Prentice Hall, 2005.

[21] J. Bollinger, *Bollinger on bollinger bands*. McGraw-Hill, 2002.

[22] H. Wickham, "Tidy data," *Journal of Statistical Software*, vol. 59, no. 10, pp. 1–23, 2014, doi: 10.18637/jss.v059.i10.

[23] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th python in science conference*, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[24] AKShare, "AKShare documentation." 2026. Available: https://akshare.akfamily.xyz/

[25] Tushare, "Tushare pro data interface documentation." 2026. Available: https://tushare.pro/

[26] CNINFO, "CNINFO listed company information disclosure platform." 2026. Available: http://www.cninfo.com.cn/

[27] P. C. Tetlock, "Giving content to investor sentiment: The role of media in the stock market," *The Journal of Finance*, vol. 62, no. 3, pp. 1139–1168, 2007, doi: 10.1111/j.1540-6261.2007.01232.x.

[28] T. Loughran and B. McDonald, "When is a liability not a liability? Textual analysis, dictionaries, and 10-ks," *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, 2011, doi: 10.1111/j.1540-6261.2010.01625.x.

[29] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011, doi: 10.1016/j.jocs.2010.12.007.

[30] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, 2019, Available: https://arxiv.org/abs/1908.10063

[31] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," in *Proceedings of the 2014 international conference on data science and advanced analytics*, 2014, pp. 102–110. doi: 10.1109/DSAA.2014.7058068.

[32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the north american chapter of the association for computational linguistics*, 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.

[33] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Chapter 8. Appendix

8.1 Results Summary

Test or result item	Observed result	Evidence source
Frontend health check	GET /health returned {"status":"ok"}	docs/frontend-chatbot.md
Chinese chatbot run	/chat returned llm.status="ok" and Chinese answer; PDF figure redacts volatile market numbers	docs/assets/frontend-chatbot-zh.png; submission/final-report/assets/frontend-chatbot-zh-redacted.png
English chatbot run	/chat returned llm.status="ok" and English answer; PDF figure redacts volatile market numbers	docs/assets/frontend-chatbot-en.png; submission/final-report/assets/frontend-chatbot-en-redacted.png
Browser rendering	Input, submit button, answer card, key points, evidence cards, and disclaimer rendered	docs/frontend-chatbot.md
Large-scale Query Intelligence evaluation	10,000 total queries; finance recall 0.975; OOD rejection 0.9729; Recall@10 0.9664; NDCG@10 0.8872	submission/final-report/evidence/large_scale_query_intelligence_eval.md
Large-scale evaluation limitation	Clarification recall 0.75, below the 0.90 target	submission/final-report/evidence/large_scale_query_intelligence_eval.jsor
Fuzz test matrix	32 total cases, 32 passed, 0 failed	submission/final-report/evidence/fuzz_query_intelligence_report.md
Fuzz score	Overall 10.0/10; NLU 10.0/10; Retrieval 10.0/10; Schema 10.0/10	submission/final-report/evidence/fuzz_query_intelligence_report.md
Representative NLU case	Moutai query normalized, entity resolved to 600519.SH, risk flag detected	submission/final-report/evidence/fuzz_query_intelligence_report.json
Representative retrieval case	News, market API, fundamental SQL, and industry SQL executed	submission/final-report/evidence/fuzz_query_intelligence_report.json

8.2 Program Documentation Walkthrough

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Start the local chatbot:

```
export DEEPSEEK_API_KEY="your_deepseek_api_key_here"
python scripts/launch_chatbot.py
```

3. Open the browser demo:

```
http://127.0.0.1:8765/
```

4. Test representative queries:

你觉得中国平安怎么样？
What do you think about Ping An Insurance (601318.SH)?
Write me a poem

Expected behavior: financial questions return evidence-grounded answers with key points, source cards, and disclaimer; English questions return English output; non-financial questions are rejected as out of scope.

8.3 Program Coding Index

The full source code should be submitted separately in the programs submission. This PDF includes a module index rather than full source listing to keep the report readable.

Module	Key paths	Purpose
Frontend	query_intelligence/api/app.py, query_intelligence/chatbot.py, scripts/launch_chatbot.py	FastAPI app, browser HTML, /chat, DeepSeek/fallback response
NLU & Retrieval	query_intelligence/nlu/, query_intelligence/retrieval/, query_intelligence/service.py, query_intelligence/contracts.py	Query normalization, classification, entity resolution, source planning, retrieval, schema contracts
Numerical Analysis	query_intelligence/retrieval/market_a query_intelligence/integrations/, data/runtime/	Market/fundamental/macro rows, technical indicators, analysis summary
Text Analysis	sentiment/schemas.py, sentiment/preprocessor.py, sentiment/classifier.py, manual_test/run_sentiment_test.py	Retrieved-document filtering, language detection, sentence segmentation, FinBERT sentiment
LLM Summary & Prediction	scripts/llm_response.py, query_intelligence/chatbot.py, config/app_config.json	Evidence packing, answer JSON, follow-up question prediction, disclaimer, model configuration
Training and Evaluation	training/, scripts/run_test_suite, evaluation/, tests/	Classical model training, regression tests, fuzz reports

The program and data submission should include the following items:

Submission item	Included material
Raw data	Runtime data in data/runtime/; external dataset cache and training assets if required by Moodle.
Programs	Repository source code excluding .env, tokens, caches, __pycache__/, temporary files, and local logs.
Model artifacts	Shipped clone-usable model files in models/*.joblib.
Documentation	README files, module documentation, evaluation reports, and this final report PDF.